Anemia Detection using Complete Blood Count (CBC) Parameters

🎯 Project Objective

Build a machine learning pipeline that uses CBC test results to predict whether a patient is anemic or not, with special focus on maximizing Recall (catch as many anemic patients as possible).

You are given a dataset with the following columns (features):

`ID, WBC, LYMp, MIDp, NEUTp, LYMn, MIDn, NEUTn, RBC, HGB, HCT, MCV, MCH, MCHC, RDWSD, RDWCV, PLT, MPV, PDW, PCT, PLCR`

📂 Part 0 – Setup

https://www.kaggle.com/datasets/ahmedelsayedtaha/complete-blood-count-cbc-test/data

Task 0.1 – Import libraries & load data

- **Import basic libraries:**
  - `pandas`, `numpy`, `matplotlib.pyplot`, `seaborn`
  - `sklearn` **modules as needed later**
- **Load the dataset into a DataFrame named** `df`**.**

🔍 Part 1 – Basic Data Exploration

Task 1.1 – First look

- **Print:**
  - `df.head()`
  - `df.tail()`
  - `df.shape` → **rows & columns**
  - `df.columns`
  - `df.info()` → **data types & non-null counts**

Task 1.2 – Descriptive statistics

- **Use** `df.describe()` **for all numeric columns.**
- **Comment on:**
  - **Which variables have very different scales?**
  - **Which variables look like counts / percentages / ratios?**

Task 1.3 – Missing values analysis

- **Calculate missing values:**

    - `df.isna().sum()`
    - **Percentage of missing values per column.**
- **Present results in a sorted table (highest missing % first).**
- **Optional: Visualize missingness with a heatmap (`seaborn.heatmap(df.isna())`).**

Deliverable: Short written summary (2–3 paragraphs) about the shape, types, and missingness of the dataset.

🩸 Part 2 – Create the Anemia Target Column

We will create a binary label: **1 = Anemic**, **0 = Not Anemic**

Given normal ranges:

- **HGB:** 11.0 to 16.0 g/dL
- **HCT:** 36.0 to 48.0 %

For this project, define anemia as:

> **Anemia = 1** if `HGB < 11.0` **OR** `HCT < 36.0`, else `0`.

Task 2.1 – Create `Anaemia` column

Add a new column:

```
df['Anaemia'] = np.where((df['HGB'] < 11.0) |
(df['HCT'] < 36.0), 1, 0)
```

-

Check value counts:

```
df['Anaemia'].value_counts()

df['Anaemia'].value_counts(normalize=True)
```

-

Task 2.2 – Class imbalance check

- Report:
    - Number of anemic vs non-anemic patients.
    - Percentage of each class.
- Visualize with a bar plot of class distribution.

Deliverable: Comment on whether the dataset is **imbalanced** or not.

📊 Part 3 – In-Depth EDA

3.1 Univariate Analysis

Task 3.1.1 – Distribution of features

For each numeric feature (except ID):

- Plot histogram + KDE (e.g. `sns.histplot(..., kde=True)`).
- Calculate **skewness** and **kurtosis**:
  - `df[num_cols].skew()`
  - `df[num_cols].kurtosis()`

Task 3.1.2 – Interpret

- Identify:
  - Highly skewed variables (e.g. |skew| > 1).
  - High kurtosis (heavy tails).
- Write down which transformations might help (e.g. log, sqrt).

3.2 Outlier Detection

Task 3.2.1 – Boxplots

- For key RBC-related features:
  `RBC, HGB, HCT, MCV, MCH, MCHC, RDWSD, RDWCV, PLT, MPV`
- Plot boxplots split by class if possible:

- `sns.boxplot(x='Anaemia', y='HGB', data=df)`

- Visually inspect for outliers.

Task 3.2.2 – IQR-based outliers

- For at least 3–5 important features (e.g. HGB, HCT, MCV, PLT, RDWCV):
  - Compute Q1, Q3, IQR.
  - Count data points outside [Q1 – 1.5*IQR, Q3 + 1.5*IQR].
- Decide:
  - Will you **cap** outliers (winsorization), **remove** them, or **keep** them (justify your choice)?

3.3 Bivariate Analysis with Target

Task 3.3.1 – Compare means between groups (t-test)

- For each key feature (e.g. `RBC, HGB, HCT, MCV, MCH, MCHC, RDWCV, PLT`):
  - Split data into:
    - `group0 = df[df['Anaemia']==0][feature]`
    - `group1 = df[df['Anaemia']==1][feature]`

    - Perform **independent t-test** (`scipy.stats.ttest_ind`).

- Record:
    - Test statistic, p-value

- Interpret:
    - Which features differ **significantly** between anemic and non-anemic?

Task 3.3.2 – Correlation analysis

Use appropriate correlation measures:

1. **Pearson correlation (continuous vs continuous)**
    - Compute Pearson correlation matrix for numeric features:
        - `df[num_cols].corr(method='pearson')`
    - Visualize as heatmap.
    - Comment on:
        - Strongly correlated pairs (possible multicollinearity).
2. **Spearman correlation (rank-based)**
    - `df[num_cols].corr(method='spearman')`

- Compare differences with Pearson.

3. **Point-biserial correlation (continuous vs binary target)**
   - For each feature vs `Anaemia` use:
     - `scipy.stats.pointbiserialr(df['Anaemia'], df[feature])`
   - Rank features by absolute correlation with `Anaemia`.
   - Identify top features that are most associated with anemia.

Deliverable:
Short report (bullets) summarizing:

- Which features are most associated with anemia.
- Any redundant / highly collinear features.

🧼 Part 4 – Data Cleaning & Scaling

Task 4.1 – Handle missing values

- Decide strategy:
  - Drop rows?
  - Impute with mean/median?

- Use appropriate imputer:
  - `SimpleImputer` (mean/median) or `KNNImputer` for numeric columns.
- Save a **clean version** of the dataset:
  - `df_clean`

Task 4.2 – Feature selection for modeling

- Drop:
  - `ID` (identifier, not useful).
- Define:
  - `X` = all feature columns (without `Anaemia`, `ID`)
  - `y` = `Anaemia`

Task 4.3 – Train-test split

- Use 70–80% train, 20–30% test.
  - `train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)`

Task 4.4 – Scaling

- For models that need scaling (Logistic Regression, KNN), use:
    - `StandardScaler` (or `MinMaxScaler`)
- **Important:** Fit scaler on **train** only, transform both train and test.

🤖 Part 5 – Model Building

You must build the following classifiers:

1. **Logistic Regression**
2. **Decision Tree**
3. **Random Forest**
4. **XGBoost**
5. **K-Nearest Neighbors (KNN)**

Task 5.1 – Baseline models

For each model:

- Train on training data.
- Predict on test data:
    - `y_pred` (class labels)
    - `y_proba` (probabilities, if available)

Task 5.2 – Evaluation metrics

Compute for **each model**:

- Accuracy
- Precision
- Recall
- F1-score
- **Matthews Correlation Coefficient (MCC)**
- ROC-AUC score

You can use:

- `classification_report`
- `confusion_matrix`
- `roc_auc_score`
- `matthews_corrcoef`

Task 5.3 – Confusion matrix

- Plot confusion matrix for each model:

    - Use `sklearn.metrics.ConfusionMatrix`

- Specifically track:

    - **False Negatives (FN)** = anemic patients predicted as non-anemic → clinically risky.

- **Goal:** Minimize FN → Maximize **Recall**.

📈 Part 6 – ROC Curve and Threshold Tuning (Maximize Recall)

Task 6.1 – ROC Curves

- For at least the **top 3 models** (e.g., Logistic Regression, Random Forest, XGBoost):

    - Compute `fpr, tpr, thresholds = roc_curve(y_test, y_proba)`.

    - Plot ROC curve.

    - Compare ROC-AUC scores.

Task 6.2 – Threshold tuning to maximize Recall

- For the **best probabilistic model**:

    - Instead of using default 0.5 threshold:

        - Loop through possible thresholds (e.g. 0.1 to 0.9).
        - For each threshold:
            - Convert probabilities to labels.
            - Compute Recall, Precision, F1.
    - Find a threshold that gives:
        - **High Recall** with acceptable Precision.
- Show a **plot**:
    - Threshold vs Recall
    - Threshold vs Precision (optional on same plot)

Deliverable:
Explain in words:

> "We choose threshold X because it gives Recall of Y (Z FN), which is important for medical diagnosis even though Precision drops to P."

🏆 Part 7 – Choose Best Model & Feature Selection

Task 7.1 – Model comparison table

- Create a table (DataFrame) where:

    - Rows = Models
    - Columns = Accuracy, Precision, Recall, F1, MCC, ROC-AUC
- Highlight:
    - Best model for **Recall** (primary)
    - Best model for **ROC-AUC** (secondary)

Task 7.2 – Final model selection

- Choose **one final model** (e.g. Random Forest / XGBoost / Logistic).
- Retrain it **on the full training data** (possibly train+test later as an extension).

Task 7.3 – Feature importance / selection

Depending on model:

- If tree-based (Random Forest / XGBoost):
    - Extract feature importances.
    - Sort in descending order.
- If Logistic Regression:

    - Look at absolute value of coefficients.

Now:

- Select top **N important features** (e.g. N = 5 or 8).
- Create a **new reduced feature set** `X_reduced` using only these features.
- Retrain the same final model on `X_reduced`.
- Re-evaluate metrics and compare with full feature model.

Question for student:
Does performance stay similar with fewer features?
If yes, this is a simpler, more interpretable model.

💾 Part 8 – Save Clean Dataset & Final Model

Task 8.1 – Save cleaned dataset

- Save `df_clean` (with `Anaemia` column included, cleaned of missing/outliers as per your decision):
    - `df_clean.to_csv('cbc_anaemia_clean.csv', index=False)`

Task 8.2 – Train final model on full data

- Use the **best algorithm** and **selected features**.
- Train on entire dataset (optionally after cross-validation).

Task 8.3 – Save model

Use `joblib` or `pickle`:

```
import joblib

joblib.dump(final_model, 'anaemia_model.pkl')

joblib.dump(scaler, 'scaler.pkl')  # if used
```

-

🧪 Part 9 – Load Model & Predict on a Sample

Task 9.1 – Load model

In a **new script or notebook cell**:

```
import joblib

import pandas as pd


model = joblib.load('anaemia_model.pkl')

scaler = joblib.load('scaler.pkl')  # if scaling used
```

Task 9.2 – Create a sample patient

- Create a one-row DataFrame with realistic values for all required features (or only selected features if using reduced model).

Example structure (values to be chosen by student):

```
sample = pd.DataFrame({
    'WBC': [7.5],
    'LYMp': [25.0],
    'MIDp': [5.0],
    'NEUTp': [60.0],
    'LYMn': [2.0],
    'MIDn': [0.5],
    'NEUTn': [4.0],
    'RBC': [3.9],
    'HGB': [10.5],
    'HCT': [34.0],
    'MCV': [88.0],
    'MCH': [28.0],
    'MCHC': [33.0],
```

```
    'RDWSD': [42.0],
    'RDWCV': [14.0],
    'PLT': [250.0],
    'MPV': [9.0],
    'PDW': [12.0],
    'PCT': [0.20],
    'PLCR': [20.0]
})
```

- Apply the same transformations:
  - Column selection.
  - Scaling (`scaler.transform(...)`) if used.

Task 9.3 – Predict

- Get:
  - Predicted class (`0 = Not anemic`, `1 = Anemic`)
  - Predicted probability.

Example:

```
sample_scaled =
scaler.transform(sample[selected_features])  # if
using scaling + selected features

pred_class = model.predict(sample_scaled)

pred_proba = model.predict_proba(sample_scaled)[:,
1]
```

- Print:

  “Predicted anemia status: 1 (Anemic) with probability 0.87”